TraceDiag: Adaptive, Interpretable, and Efficient Root Cause Analysis on Large-Scale Microservice Systems

Ruomeng Ding
Chaoyun Zhang
Microsoft
China

Lu Wang
Yong Xu
Microsoft
China

Minghua Ma
Xiaomin Wu
Microsoft
China

Meng Zhang
Qingjun Chen
Microsoft 365
China

Xin Gao
Xuedong Gao
Microsoft 365
China

Hao Fan
Microsoft 365
China

Saravan Rajmohan
Microsoft 365
USA

Qingwei Lin
Dongmei Zhang
Microsoft
China

ABSTRACT

Root Cause Analysis (RCA) is becoming increasingly crucial for ensuring the reliability of microservice systems. However, performing RCA on modern microservice systems can be challenging due to their large scale, as they usually comprise hundreds of components, leading significant human effort. This paper proposes TraceDiag, an end-to-end RCA framework that addresses the challenges for large-scale microservice systems. It leverages reinforcement learning to learn a pruning policy for the service dependency graph to automatically eliminates redundant components, thereby significantly improving the RCA efficiency. The learned pruning policy is interpretable and fully adaptive to new RCA instances. With the pruned graph, a causal-based method can be executed with high accuracy and efficiency. The proposed TraceDiag framework is evaluated on real data traces collected from the Microsoft Exchange system, and demonstrates superior performance compared to state-of-the-art RCA approaches. Notably, TraceDiag has been integrated as a critical component in the Microsoft M365 Exchange, resulting in a significant improvement in the system's reliability and a considerable reduction in the human effort required for RCA.

CCS CONCEPTS

• **Computing methodologies** → **Sequential decision making**.

KEYWORDS

Trace data, Root Cause Analysis, Reinforcement Learning

ACM Reference Format:

Ruomeng Ding, Chaoyun Zhang, Lu Wang, Yong Xu, Minghua Ma, Xiaomin Wu, Meng Zhang, Qingjun Chen, Xin Gao, Xuedong Gao, Hao Fan, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. TraceDiag: Adaptive, Interpretable, and Efficient Root Cause Analysis on Large-Scale Microservice Systems. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23), December 3–9, 2023, San Francisco, CA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3611643.3613864

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ESEC/FSE '23, December 3–9, 2023, San Francisco, CA, USA
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0327-0/23/12…$15.00
https://doi.org/10.1145/3611643.3613864

1 INTRODUCTION

The microservices architecture has gained widespread popularity in modern production systems that leverage cloud technologies [46]. This approach involves decomposing large systems into smaller, self-contained components that communicate with each other, thereby enabling agile development and deployment [10, 21]. For instance, Twitter has about 1,200 microservices on the server side [29] and Microsoft Exchange employs hundreds of microservices to enable email delivery, with each microservice interacting with others based on complex dynamic dependencies [56]. In such intricate systems, anomalous incidents may occur in individual components, thereby impacting downstream components and resulting in unexpected end-to-end latency and a high number of delayed email deliveries [20]. These issues can significantly harm the customer experience, leading to substantial financial losses [25], and adversely affect the reputation of service providers.

To ensure the reliability of cloud services, on-call engineers rely on a range of system data sources such as key performance indicators [28], logs [23], and traces [56] to develop automated and intelligent incident detection and root cause analysis (RCA) solutions [27, 50, 56, 61, 62]. The goal is to quickly mitigate incidents and minimize their impact on business operations by analyzing anomaly propagation chains to pinpoint the root causes. However, this is a challenging task, particularly given the complex interactions and interdependencies among the system components, which is further compounded by the scale of modern cloud systems. Research shows that RCA can take several hours on average without the use of automated tools, making it a time-consuming and arduous process [20, 52]. In large systems like Microsoft Exchange, where service performance would be influenced by multiple factors including infrastructure scale, automatic scaling, dynamic load balancing, system updates, and security measures [6, 13], RCA becomes even more intricate.

Traces serve as a crucial component of system observability, capturing microservice invocation dependency and associated performance metrics. Graph analysis techniques are commonly employed in trace analysis to automatically localize faults by traversing service dependency graphs and detecting anomalies based on service quality metrics [5, 13, 48, 63]. However, the scale of modern

microservice systems can lead to challenges in accurate anomaly detection and efficient graph traversal, as they generate massive volumes of system traces containing hundreds to thousands of components and their dependencies. One contributing factor to the complexity of RCA is the presence of redundant components that have no impact on the incident and can be safely removed without compromising the RCA performance [20, 55]. Fig. 1 illustrates an example trace, depicting the latency evolution of each component. The root cause component F experiences a surge in latency, subsequently affecting component C and the frontend node. However, other components that do not interact with F maintain normal latency behavior and do not contribute to the RCA process. These components can be considered "redundant" and eliminating them from the trace can reduce noise and enhance the efficiency of RCA.

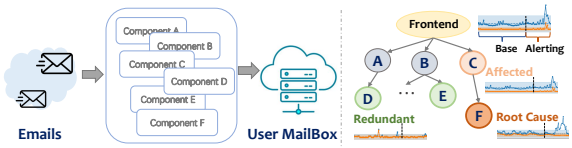


Figure 1: An example of trace in Microsoft Exchange system.

Pruning redundant components from dependency graphs is a widely adopted technique in the RCA process [20, 55]. However, determining which components to prune poses a significant challenge. Aggressive pruning, where too many components are removed, runs the risk of accidentally removing the root cause node, rendering further RCA impossible. Conversely, a lenient pruning strategy that preserves too many components can result in inefficient RCA and generate noisy outcomes. Current pruning policies are often impromptu, based on engineers' expertise and tailored to specific incidents, lacking interpretability and susceptible to system changes over time. Consequently, significant human effort is required, and a lack of standardized and unified rules persists. Therefore, there is a pressing need for an adaptive service graph pruning policy that can offer interpretability in the pruning process.

To address the aforementioned challenges, this paper introduces TraceDiag, a novel framework for RCA in large-scale microservice systems. TraceDiag leverages reinforcement learning (RL) to acquire an automated, interpretable, and adaptable pruning policy that effectively removes redundant components, enhancing the efficiency and accuracy of the RCA process. The policy is based on graph pruning rules derived from experienced engineers and comprehensive trace analysis, ensuring domain knowledge and industry best practices are incorporated. The pruned service graph is then utilized for RCA using causal methods. The practicality and effectiveness of TraceDiag have been demonstrated through its deployment as a core RCA engine in a Microsoft microservice system. In summary, the contributions of this paper include:

- We introduce TraceDiag, an end-to-end RCA framework for large-scale microservice systems. The TraceDiag integrates graph pruning to enhance RCA efficiency and a causal RCA method that surpasses correlation-based approaches to provide more accurate and robust results.
- We employ RL to automate the graph pruning by selecting from a pre-defined pruning action pool designed through comprehensive trace analysis. This ensures interpretability and adaptability to new RCA instances.
- We evaluate our proposed framework on real data traces collected from the Microsoft Exchange system, demonstrating superior performance compared to state-of-the-art RCA approaches.
- The proposed framework has been integrated as a critical component in the Microsoft M365 Exchange system, leading to 75.1% higher RCA accuracy and reduce time for RCA by up to 96.5%. This yields a significant improvement in the system's reliability and considerable reduction for the human effort for RCA.

To the best of our knowledge, our TraceDiag is the first to apply RL as the trace pruning process for the RCA of microservice systems.

2 BACKGROUND AND MOTIVATION

This section provides an overview of the Microsoft Exchange Microservice System, which serves as the experiment and implementation of the TraceDiag approach. We also formulate the problem of trace-based RCA in this section.

2.1 Microsoft Exchange Microservice System

Microsoft Exchange is a cloud-based email delivery system that offers an efficient and reliable message routing and delivery service. Email delivery latency is a crucial performance metric that constitutes the primary service level agreement provided to customers. The message delivery process in the Microsoft Exchange cloud system involves numerous components from various servers and services, processing billions of emails daily. We illustrate the system in the left of Fig. 1.

To analyze email delivery latency in Microsoft Exchange, an email request generates a trace that captures its invocation dependencies. The trace is represented as a directed tree structure, denoted as $\mathcal{G}$, where nodes correspond to microservices or functions invoked, and edges represent their interdependent relationships [56]. Due to the interdependent nature of the edges in $\mathcal{G}$, they can depict the causal relationship between microservices, which goes beyond simple correlation. The trace also records relevant information, such as structural attributes, service metadata, call latency, and temporal information, for subsequent analysis. The trace data, coupled with the latency evolution of each microservice, constitute multivariate time series [57–60] with a structural topology [1, 32, 54]. Note that in contrast to previous studies such as [31, 35], we have access to and knowledge of the trace topology. As a result, there is no need for the causal discovery [11, 24, 53] step to construct the service dependency graph in this work.

Each email request incurs end-to-end latency that includes delays from all the microservices in the trace. If the latency exceeds the established service level agreement (SLA), it can result in slow email deliveries, leading to customer dissatisfaction and escalation. To detect anomalous latency surges, Microsoft Exchange employs TraceArk [56], and subsequently performs RCA to localize problematic components that contribute to the anomaly. RCA plays a critical role in identifying the cause of the issue, enabling prompt mitigation actions, and minimizing the impact on business.

2.2 Root Cause Analysis on Trace

Incidents occurring in the microservice system frequently lead to a considerable number of emails that fail to meet the latency Service Level Agreement (SLA) or experience a noticeable increase in end-to-end email delay, thereby negatively impacting the user

experience. To capture the overall system's end-to-end delay, we introduce a frontend node denoted as X_f, which encompasses the time interval from when an email is sent to when it is received. This particular latency metric is closely monitored by engineers, as it serves as an indicator of the system's reliability performance.

Once anomalous events are detected, specifically when the frontend node X_f demonstrates high latency, the primary goal is to identify the microservices accountable for the anomaly and promptly mitigate it [9]. The RCA process aims to identify the root cause nodes within the trace $\mathcal{G}$ that are most likely to have a significant impact on the end-to-end email delay. By minimizing the time needed to pinpoint the root cause of an anomaly, RCA plays a crucial role in ensuring the high reliability of microservice systems.

Let $\mathbf{X} = (X_1, X_2, \cdots, X_n)$ represent the set of components in the service dependency graph $\mathcal{G}$, where each component collects the latency metric within a specific time window. The objective of the RCA process is to identify the subset of nodes $\mathbf{c} \subseteq (1, \cdots, n)$ that are most likely to be the root cause of the incident. We define the duration of the anomalous event as the alerting window, denoted as W_a. Additionally, latency data collected from a base window W_b, when the system is operating normally, will be utilized for comparative analysis for the RCA, as shown in the right of Fig. 1.

3 EMPIRICAL STUDY ON TRACE DATA

In order to facilitate the comprehension of the RCA framework, it is necessary to examine the trace data more closely and characterize the latency behavior of the root cause service and healthy components in the system during an incident. Insights gained from the analysis can inform the design of the pruning policy and subsequent RCA process. We conduct a brief empirical analysis of historical incident instances in the Microsoft Exchange system, where one or more services caused problems leading to incidents.

The empirical study utilizes a trace dataset obtained from the Microsoft Exchange microservice system, specifically focusing on 12 instances of RCA referred to as incidents. In these incidents, a subset of services within the system exhibits high latency, leading to unexpected delays in end-to-end processing. Engineers have already identified the root causes for each of these RCA instances. The dataset covers a duration of 345 days and includes trace data from over 708 distinct microservices. These microservices are distributed across 11 forests, resulting in a cumulative count of 8,424 microservices. The substantial size of the dataset facilitates a comprehensive analysis of trace characteristics and the distinctive properties of the root cause service.

Building upon the methodology outlined in [56], we calculate two types of latency measurements for each component in the trace: *(i)* Exclusive Latency (ExL), which indicates the running time of the component itself, and *(ii)* Inclusive Latency (InL), which is the difference between the component entry and exit time. The InL metric is equal to the sum of the component's exclusive latency and the inclusive latency of its children. Both ExL and InL metrics are frequently used in trace studies [15, 22, 34]. In addition, we define a component as being *"affected"* by the root cause service only if the root cause is a descendant of this node. This means that the node directly or indirectly invokes the root cause service and is thus affected. The remaining components that do not interact with the root cause service are considered *"redundant"*.

Table 1: The statistics of trace data during incidents.

Metric	Total Node	Total Edge	Root Cause Num.	Affected Nodes
Mean	549	29,534	3	16
Median	561	31,271	1	9

3.1 Redundant Components for RCA

We begin by computing the statistics of the trace data and the root cause components during incidents, as presented in Table 1. Metrics are computed over all incident instance, similarly hereinafter. It is noteworthy that the scale of the microservice system is immense, with an average of over 500 components and approximately 30,000 invoked edges, making the RCA process highly complex. However, despite the large number of services, only a few components are responsible for causing the incidents, and only a small subset of components is affected by the root cause node. This implies that a significant number of components can be deemed redundant for the RCA process and hence can be pruned.

Takeaways 1: A limited number of components are accountable for causing the incidents, and the impact of the root cause node is constrainted to a small subset of components, resulting in redundancy in the RCA process.

3.2 Exclusive Latency of Root Cause Services

Table 2: Exclusive latency statistics of root cause and non-root cause components.

Service	Metric	ExL [s]	Percentile
Root Cause	Mean	50.798	P96
	Median	1.543	P96
Non-root cause	Mean	16.002	\
	Median	0.053	\

Furthermore, we perform a comparison of the exclusive latency between the root cause services and non-root cause components, as shown in Table 2. The objective of this analysis is to determine if ExL can effectively distinguish between these two types of components. By using ExL instead of InL, we eliminate the contribution of latency introduced by other services invoked by the target component [56]. Observe that the root cause components exhibit an average ExL that is more than three times higher than that of non-root cause components. Moreover, during an incident, the ExL of the root cause components exceeds that of over 96% of all components on average. This significant difference in ExL values establishes it as a robust indicator for identifying the true root cause. We can then utilize ExL to design additional pruning actions aimed at enhancing the efficiency of the RCA process, as will be elaborated in Sec. 4.3.1.

Takeaways 2: The service responsible for the root cause of an incident typically shows a high exclusive latency, exceeding that of non-root-cause components.

3.3 Root Cause vs. Anomaly

Next, we analyze the correlation between the anomalous components and the root cause service when incidents occur. However, it is important to note that an anomalous component does not necessarily imply that it is the root cause of the incident, as it may

Table 3: Percentiles of anomaly indicators and identification ratio by TraceArk of root causes and affected components.

Service	Metric	NormalizeCount	RankScore	OverHead	Anomaly Ratio
Root Cause	Mean	P85	P97	P94	90.48%
	Median	P83	P99	P99	
Affected	Mean	P91	P92	P93	37.49%
	Median	P90	P99	P99	

have been impacted by another service that it calls [13]. To identify anomalies within the microservice system at the service level, we employ a state-of-the-art anomaly detector called TraceArk [56]. This detector has been integrated into Microsoft Exchange and serves as a prerequisite for the functioning of our TraceDiag. TraceArk provides three intermediate metrics as indicators of anomaly, namely: *(i)* **NormalizeCount**, quantifying the number of occurrences of a component normalized by the total number of traces; *(ii)* **OverHead**, referring to the increment in terms of the total latency of the alerting window with respect to the base window for comparison; and *(iii)* **RankScore**, indicating the degree of anomaly in the temporal dimension based on the continually in the anomaly and numerical changes derived from k-sigma threshold [3]. Higher values of these metrics indicate a greater likelihood that the component is anomalous. Additionally, TraceArk provides a direct label to predict whether a component is anomalous based on the aforementioned metrics.

Table 3 shows the percentile statistics of *NormalizeCount*, *RankScore*, and *OverHead*, and the anomaly identification ratio by TraceArk, for both the root cause service and the affected components. Percentiles are computed among all components for individual incidents. Notably, both the root cause and affected services exhibit high percentile values in terms of the three anomaly indicators. This implies that these services are very likely to be identified as anomalies. The high anomaly identification ratio by TraceArk further supports this observation, with over 90% of root cause nodes being identified as anomalies, while this ratio is much lower for affected services. This is reasonable, as other services may be impacted to different degrees, and some may not exhibit strong anomaly behaviors. Based on these insights, we leverage these anomaly indicators to prune irrelevant components, as detailed in Sec.4.3.2.

Takeaways 3: The anomaly indicators in TraceArk show that the root cause and other services impacted by it are highly likely to have high values that indicate to anomaly. The root cause service has a higher probability of being identified as an anomaly compared to other affected components.

3.4 Latency Correlation Analysis

Finally, we conducted an analysis of the latency similarity between the end-to-end delay observed at the frontend node and the root cause service, as well as the services affected by calculating their mutual Pearson correlation coefficients. This analysis aimed to determine whether the root cause and affected services exhibited similar latency patterns in comparison to the end-to-end delay.

The statistical results are presented in Table 4. It can be observed that, on average, both the root cause service and the affected services displayed Pearson correlation coefficients of over 0.5 with

Table 4: The Pearson correlation coefficient statistics between InL of the fontend node and root cause/affected services.

Service	Metrics	Pearson Coefficient	Percentile
Root Cause	Mean	0.598	P83
	Median	0.626	P92
Affected	Mean	0.526	P81
	Median	0.519	P88

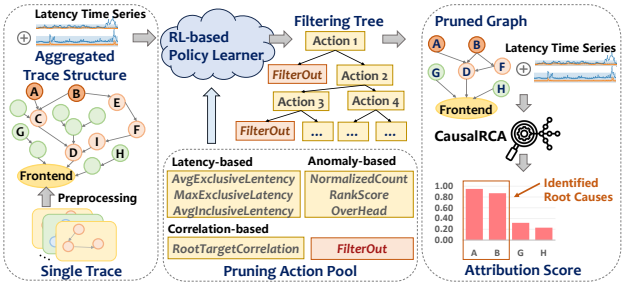


Figure 2: The overall framework of TraceDiag.

the frontend node, indicating a strong similarity in latency patterns. Additionally, we noted that the coefficients of the root cause services, on average, exceeded those of approximately 83% of the components in the entire system. While this percentile was slightly lower for the affected services (81%), the Pearson correlation coefficients of both the root cause and affected services ranked highly among all components. This valuable insight serves as the basis for designing an additional pruning action that considers the latency similarity with the end-to-end delay, as described in Sec. 4.3.3.

Furthermore, we observed only 7.1% of the components with the highest Pearson correlation coefficient with the end-to-end delay, as monitored at the frontend node, were indeed the root cause of the incident. This finding highlights the fact that a high correlation does not necessarily indicate causation. Consequently, we are motivated to adopt a causal approach [38], as opposed to correlation-based methods, for the subsequent RCA phase. This shift in approach aims to achieve a more robust and accurate RCA process. We provide a detailed explanation of this approach in Sec. 4.5.

Takeaways 4: The inclusive latency of both the root cause service and other affected services typically show high correlations with the overall end-to-end delay. However, the component with the highest latency correlation with the frontend node may not necessarily be the root cause of the incident.

4 THE DESIGN OF TRACEDIAG

We overview the architecture of TraceDiag in Sec. 4.1, and delve into the details of its components in the subsequent sections.

4.1 TraceDiag in a Nutshell

Fig. 2 illustrates the overall framework of TraceDiag for the RCA process applied to trace data. The Microsoft Exchange system collects a large volume of single trace data, each containing the invocation sequence and latency information of services involved in email delivery. These traces are preprocessed and aggregated to construct a dependency graph comprising hundreds of components.

Table 5: The list of pruning action pool employed in TraceDiag, and their thresholds.

Pruning Action	Type	Threshold
AvgExclusiveLentency	Latency	*P*80, *P*85, *P*90, *P*95, *P*99
MaxExclusiveLentency	Latency	0.01*s*, 0.05*s*, 0.1*s*, 0.5*s*, 1*s*
AvgInclusiveLentency	Latency	*P*80, *P*85, *P*90, *P*95, *P*99
NormalizedCount	Anomaly	*P*50, *P*65, *P*70, *P*80, *P*90
OverHead	Anomaly	*P*80, *P*85, *P*90, *P*95, *P*99
RankScore	Anomaly	*P*80, *P*85, *P*90, *P*95, *P*99
RootTargetCorrelation	Correlation	0.1, 0.3, 0.5, 0.7, 0.9
FilterOut	–	–

The next stage involves pruning redundant components from the dependency graph to enhance the efficiency of the subsequent RCA process. We define a library of pruning actions based on the analysis presented in Sec. 3 and employ RL to automatically learn an interpretable and adaptive pruning policy from historical incidents. This learned policy is then applied to the complete service dependency graph, resulting in a simplified structure with significantly fewer components. It is important to ensure that the parent and child nodes of a pruned node are automatically connected to maintain a coherent topology structure. In Figure 1, component *E* is pruned by the filtering tree, resulting in its parent *B* and child *F* being connected in the pruned graph. Further details about the pruning process can be found in Sec. 4.3 and 4.4.

Finally, we perform RCA using a causal approach on the pruned graph, taking into account the evolution of service latencies, to identify the root cause of the incident and attribute its responsibility. A detailed explanation of the RCA process is provided in Sec. 4.5. In the subsequent subsection, we delve into the specifics of each component within the TraceDiag framework.

4.2 Trace Preprocessing

The Microsoft Exchange system generates gigabytes of trace data daily, which are collected and stored in a database for analysis. However, performing RCA on such a large amount of data is impractical and unnecessary due to the huge computing resource required, and the redundancy present in the full data. To mitigate these issues, we first randomly sample 1% of the raw trace to maintain RCA overhead without compromising RCA performance. Note that the sampling is performed at the trace level, not at the component level. Therefore, a component is unlikely to be eliminated from the topology if it is frequently invoked. After trace sampling, we aggregate the time series for each component using the average function, normalized by the appearance of traces invokes that component at the 15-minute level. This approach offers a good tradeoff between processing overhead and RCA performance.

4.3 Service Dependency Graph Pruning

Building on the insights obtained in Sec.3.1, it is imperative to eliminate redundant nodes from the complete trace data as most of them are not affected by problematic services, to improve the following RCA efficiency. To this end, we propose three sets of interpretable pruning actions tailored to the trace data based on findings from Sec. 3: *(i)* latency-based pruning, *(ii)* anomaly-based pruning, and *(iii)* correlation-based pruning. We present the complete set of pruning actions utilized in TraceDiag in Table 5, and the subsequent subsections provide detailed explanations. These pruning actions help to remove redundant nodes while minimizing the risk of dropping true root cause and affected nodes from the service dependency graph. By doing so, the RCA efficiency can be significantly improved while maintaining high accuracy.

4.3.1 Latency-Based Pruning. Based on the findings discussed in Sec. 3.2, where it was observed that the root cause service tends to exhibit high latency during an incident, we propose three sets of pruning actions based on this observation, as follows: *(i) AvgExclusiveLatency*, which represents the average exclusive latency of a service; *(ii) MaxExclusiveLatency*, which denotes the maximum exclusive latency of a service; and *(iii) AvgInclusiveLatency*, which indicates the average inclusive latency of a service. We include it as a pruning action for the sake of completeness. If the aforementioned metrics fall below a certain percentile or time threshold, we remove the corresponding services from the service graph, as they are unlikely to be the root cause service and are less likely to contribute to the incident.

4.3.2 Anomaly-Based Pruning. The second set of pruning actions considered in TraceDiag is based on the insights gained from Sec. 3.3, which demonstrate that both the root cause service and affected services exhibit high percentile values of anomaly indicators provided by TraceArk. We design three sets of pruning actions accordingly: *(i) NormalizedCount*, *(ii) RankScore*, and *(iii) OverHead*. These actions aim to filter out redundant components with low anomaly indicator percentile values compared to other components. Such components are unlikely to be the root cause or affected services, and therefore pose a low risk of being mistakenly removed. Consequently, they can be safely eliminated from the service graph without compromising the RCA performance.

4.3.3 Correlation-Based Pruning. Lastly, based on the insights obtained from Sec. 3.4, we have observed that there is generally a high correlation between the end-to-end delay and both the root cause service and the affected services. This indicates that they exhibit similar latency behaviors during an incident. To leverage this observation, we introduce the last pruning action, namely *RootTargetCorrelation*. This action involves evaluating the Pearson correlation coefficient between the end-to-end delay and the InL of a target node. If the correlation coefficient falls below a specified threshold, we remove the target node from the service graph. This action filters out components that demonstrate dissimilar latency patterns from the frontend node, implying that they are unlikely to be relevant to the incident.

Overall, based on the insights obtained from Sec. 3, we have developed three distinct sets of 35 interpretable pruning actions to eliminate redundant components from the service graph, considering various characteristics of the root cause in the trace data. Additionally, we have introduced a general action, called *FilterOut*, which is responsible for executing the pruning actions associated with each filter in the filtering tree. The details of the pruning tree construction is elaborated in the subsequent Sec. 4.4.

4.4 Learning Pruning Policy with RL

Once we have a comprehensive library of candidate pruning actions in Table 5, the next step is to select appropriate actions from this pool and determine their execution sequence to create a pruning

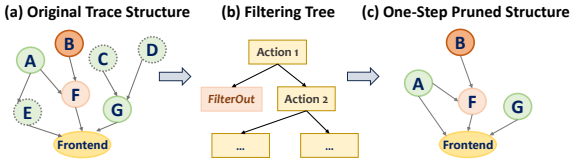


Figure 3: An example of a filtering tree and its execution for one-step pruning. C, D and E are pruned by Action 1.

policy. The objective is to maximize the effectiveness of the RCA while minimizing the number of components in the service graph, thereby improving the efficiency of the RCA process. However, designing an optimal pruning policy is not a straightforward task. Exploring all possible combinations of N actions would require a complexity of $O(2^N)$ trials. Additionally, determining the optimal execution sequence of these actions further complicates the problem. Given the diverse nature of incidents and trace data, relying solely on human expertise to design an adaptive graph pruning policy that can generalize to unseen RCA instances is infeasible, even with a candidate list of pruning actions at hand.

To overcome this challenge, we propose a solution that utilizes a RL approach to automatically learn a pruning policy, known as a filtering tree, from historical data. This learned filtering tree can then be applied to new incidents without the need for human intervention. The RL approach offers several distinct advantages:

- **Automatic Learning**: The RL approach can automatically to learn the pruning policy without requiring human intervention. This automation reduces the reliance on manual expertise and allows for efficient policy learning.
- **Effective Searching**: The RL approach naturally handles delayed rewards by employing T-D learning, enabling efficient RCA performance. Furthermore, by balancing exploration and exploitation, RL effectively explores the complex action space, resulting in significantly improved efficiency compared to exhaustive search methods.
- **Adaptability**: While the RL policy is represented via a parametric model, the learned filtering tree is adaptive and can be applied to new incidents without the need for further refinement by humans. This adaptability enhances the generalizability of the pruning policy to unseen scenarios.

We begin by introducing the concept of the filtering tree and subsequently provide a detailed description of our RL approaches in the following subsections.

4.4.1 Filtering Tree. A filtering tree is a hierarchical structure utilized to eliminate redundant components in the service dependency graph. It is structured as a binary tree, with each node representing a pruning action selected from Table 5. If a node has children, it divides the full component set in the dependency graph into subsets based on whether they satisfy the action's condition. Specifically, if a component surpasses the pruning action's threshold, it is directed to the right child node; otherwise, it is directed to the left child node. The *FilterOut* action is exclusively present in the leaf nodes of the filtering tree, where it executes the pruning action on the components traversing its branch.

In Fig. 2, we provide an example of a one-step pruning. The original trace consists of 7 components, excluding the frontend node. Components C, D, and E are assigned to the left branch of

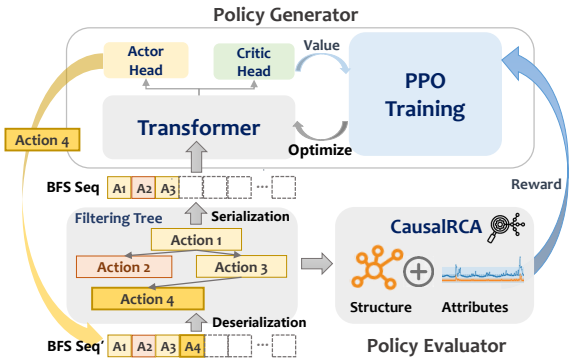


Figure 4: The RL-framework for the filtering tree training.

Action 1 and meet the *FilterOut* condition, leading to their removal. The remaining components that meet the threshold of Action 1 are directed to its right branch and subsequently evaluated by Action 2. By applying this filtering process to all service components in the dependency graph, redundant nodes are eliminated, resulting in a more efficient RCA procedure.

4.4.2 Model Overview. The overall RL-framework employed for the construction of the filtering tree is illustrated in Fig. 4. This framework consists of two main components: *(i)* a **Policy Generator**, responsible for learning a filtering tree that makes decisions regarding the selection of pruning actions and the branches to follow in order to create an effective filter; and *(ii)* a **Policy Evaluator**, which uses the RCA system to evaluate the performance of the generated filtering tree. The output of the policy evaluator serves as a reward signal, which is utilized to enhance the policy generator using Proximal Policy Optimization (PPO) techniques. Following the training process, the policy generator is capable of constructing a filtering tree that achieves both high RCA accuracy and efficiency.

4.4.3 MDP Formulation. We start by formulating the generation of the filtering tree as a finite-horizon Markov Decision Process (MDP). The environment comprises an action space $\mathcal{A}$, a state space $\mathcal{S}$, and a reward function R. The filtering tree f can be serialized as a sequence of nodes by visiting each node in a breadth-first manner, denoted as τ. This serialization enables us to represent the filtering tree as a vector and recover it through deserialization. Therefore, we define the action search space as a discrete sequence of "nodes" that correspond to pruning actions selected from Table 5.

At each time step t, given the current state s_t of the filtering tree and the pruned trace structure, the policy generator $\pi_\theta(a_t|s_t)$ selects a pruning action a_t to partition the set of components from the dependency graph. This action is considered as adding a new node to the filtering tree. Upon receiving a reward signal r_t from the policy evaluator at each step, the environment transitions to a new state, resulting in a new filtering tree and pruned dependency graph. The objective of the agent is to learn the optimal policy that maximizes the expected cumulative reward by selecting the most appropriate pruning action at each step. The different components of the MDP are defined as follows:

- **State** $s_t \in \mathcal{S}$ encompasses two components: the structure of the filtering tree and the current structure, both serialized using the breadth-first search (BFS) algorithm. The complexity of the trace,

which includes the number of nodes, edges, and sparsity, is also included as a metric in the state tensor.

- **Action** $a_t \in \mathcal{A}$ corresponds to selecting a single action from the set of actions defined in Table 5 to be added to the filtering tree.
- **Reward Function** $R(s_t, a_t) = \alpha \cdot r^{com} + \beta \cdot r^{rca}$ comprises two components. $r^{com} = -(|N| + |E| + \frac{|E|}{|N|^2})$, quantifies the current complexity of the pruned trace structure. Here, α and β denote the weights assigned to the rewards. To be Specific, we set $\alpha = 0.01$ and $\beta = 1$. $|N|$ and $|E|$ represent the number of nodes and edges, respectively. $r^{rca} = PR@Avg+RankScore$, measures the accuracy of the RCA based on the pruned trace structure. For a detailed explanation of *PR@Avg* and *RankScore*, refer to Sec. 5.1.3.

The reward function incorporates two terms, aiming to maximize RCA accuracy while minimizing graph complexity. This ensures high RCA performance while achieving efficiency improvements.

4.4.4 Filtering Tree Generator. Given the current state s_t of the filtering tree and the pruned trace structure, the filtering tree generator $\pi_\theta(a_t|s_t)$ learns to choose the next pruning action added to the filtering tree. This action is selected from the Actor head of the model in Fig. 4. Since the filtering tree is serialized as a sequence vector, we leverage the Transformer [47] as the model of π_θ to map the state s_t into action a_t.

Cascade Policy Structure: In order to effectively remove redundant components from the trace, it is essential to prioritize the *FilterOut* action. Otherwise, the resulting pruning tree would become excessively large. To address this, we introduce a cascade policy [51] that promotes the selection of *FilterOut* actions:

$$\pi_\theta(a_t|s_t) = \begin{cases} \pi_\theta(a_t^{FO} = 1|s_t), & a_t^{FO} = 1 \\ \pi_\theta(a_t^{FO} = 0|s_t) \cdot \pi_\theta(a_t^{FE}|s_t, a_t^{FO} = 0), & a_t^{FO} = 0 \end{cases} \tag{1}$$

where $a_t^{FO} = 1$ represents the decision of the policy generator to take the *FilterOut* action, while $a_t^{FO} = 0$ implies the omission of this action. On the other hand, a_t^{FE} denotes the policy generator's selection of a non-*FilterOut* pruning action. It is important to note that the *FilterOut* action leads to an early termination of the expansion process for the filtering tree in that particular branch.

4.4.5 In Situ Constraints. By formulating the tree construction problem as a sequential decision-making problem, we can effectively incorporate domain knowledge by imposing constraints directly on the search space. This allows us to define a broad range of in situ constraints, where we can determine which actions are disallowed during the traversal. To achieve this, we simply assign zero probability to any pruning action that would violate a constraint before selecting the action. This approach ensures that all generated filtering trees adhere to the specified constraints during the learning process, reducing complexity and eliminating the need for post hoc rejection of samples.

To enhance the interpretability of the resulting filtering trees, we impose several constraints: *(i)* We limit the length of each filtering tree expression to a predefined range between a minimum and maximum length, such as 2 to 30. *(ii)* We prohibit the placement of the *FilterOut* action as the right child of any nodes. *(iii)* We disallow a child node from being the same action as its parent to avoid redundant execution. *(iv)* To prevent invalid policies, we ensure that each expression includes at least one *FilterOut* action within the latest five steps, with a probability of 0.8. This probability gradually decays in subsequent learning episodes, following a decay rate of $0.8^{episodes+1}$. Furthermore, to promote diversity in the expression of the filtering tree, we introduce randomness in the first 5 episodes by randomly selecting actions with an equal probability.

4.4.6 Policy Evaluator. Once we obtain a sequence τ generated by the Policy Generator, we deserialize the corresponding filtering tree structure f. This filtering tree is then directly applied as the modular component for pruning the trace structure. To assess the effectiveness of the policy, we execute it on multiple incident instances and calculate the average episodic reward, which includes both the RCA accuracy and the complexity of the pruned trace structure. This computed reward serves as the signal for training the Policy Generator using RL. The cumulative reward function of the Policy Generator is defined as follows:

$$R(\tau) = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \alpha \cdot r_{t,i}^{com} + \beta \cdot r_i^{rca}. \tag{2}$$

4.4.7 Actor Critic Algorithm Training with PPO. The reward function derived from the Policy Evaluator is non-differentiable due to its origin from a control environment and the combination of multiple objectives. Therefore, we utilize RL to optimize the Policy Generator. In such cases, auto-regressive models with black-box reward functions often employ the Proximal Policy Optimization (PPO) algorithm [43], which optimizes the objective given by:

$$J(\theta) = E_{(s,a)\sim D_{\pi_\theta}} \left[\frac{\pi_\theta(a|s)}{\pi_{old}(a|s)} Q_\omega(s,a) \right] - \lambda KL\left[\pi_{old}|\pi_\theta\right]. \tag{3}$$

Here, π_{old} represents the policy model from previous training iterations, Q_ω denotes the Critic, which is also represented by a Transformer model with parameter ω. It is important to note that the optimization of $J(\theta)$ aims to maximize the expected reward under the distribution.

The training process is fully automated and does not require human intervention. After the training phase, we obtain a filtering tree that effectively removes redundant components for RCA. The trained filtering tree can generalize to unseen incidents, providing efficient RCA capabilities without the need for manual intervention.

4.5 Causal Root Cause Analysis

After obtaining the pruned dependency graph, we utilize it as a graphical causal model to perform RCA based on the changes in the causal mechanism of each node in case of a failure in the pruned dependency graph. This method, inspired by the research in [6], is called CausalRCA. The reason we adopt a causal method instead of correlation-based approaches is that a node that has the highest correlation with the end-to-end delay may not necessarily be the true root cause of the incident, as discussed in Sec. 3.4. This is because a component's latency is affected by the children services or functions it invokes, which may act as confounding factors. The causal method can minimize the impact of such effects and thus deliver more robust and accurate RCA performance.

We model the pruned service dependency graph using a graphical causal model. Bayesian models are employed to represent the

latency of individual components in both the base window (W_b) and the alerting window (W_a) [37]. The joint distributions of all components are denoted as $\mathbf{P}^{\mathbf{b}}_{\mathbf{X}}$ and $\mathbf{P}^{\mathbf{a}}_{\mathbf{X}}$. The Bayesian models factorize into the product of marginal distributions for each component, $\mathbf{P}_{\mathbf{X}} = \prod_{j=1}^{n} P_{X_j|PA_j}$. Here, $P_{X_j|PA_i}$ represents the causal mechanism between component X_j and its immediate parent variables PA_j, which correspond to the services invoked by X_j [39].

Root cause nodes typically display a noticeable increase in latency in the alerting window, which in turn affects their parent services. This results in a shift in the causal mechanisms of these root cause nodes. Upon the occurrence of an incident, the joint distribution $\mathbf{P}^{\mathbf{a}}_{\mathbf{X}}$ transforms by replacing the original causal mechanisms of the root cause subset $\mathbf{c}$ with new ones, *i.e.*,

$$\mathbf{P}_{\mathbf{X}} = \prod_{j \notin \mathbf{c}} P^{a}_{X_j|PA_j} \prod_{j \in \mathbf{c}} P^{b}_{X_j|PA_j}, \tag{4}$$

where $P^{b}_{X_j|PA_j}$ is the new causal mechanism of node j in the alerting window. In practice, both $P^{a}_{X_j|PA_j}$ and $P^{b}_{X_j|PA_j}$ are approximated with machine learning models from observational data available in the base window and detection window, respectively.

To evaluate the influence of each component on the frontend latency distribution P_{X_f}, we use the Shapley value [44] to measure the change in P_{X_f} when an evaluation function Φ is altered. We compare the latency distributions of the frontend node in the base window $P^{b}_{X_f}$ and detection window $P^{a}_{X_f}$ using the evaluation function Φ, such that $\Delta\Phi = \Phi(P^{a}_{X_f}) - \Phi(P^{b}_{X_f})$. In this work, we employ the median function as Φ to quantify the disparity between the latency distributions of the frontend node in the two windows. However, other suitable evaluation functions can be utilized.

5 EVALUATION

We present a thorough evaluation of TraceDiag, utilizing real production data, and comparing its performance with several state-of-the-art baselines. This section aims to address the following research questions (RQs):

- **RQ1:** How effective is TraceDiag in diagnosing the root cause of incidents occurring in large-scale microservice systems?
- **RQ2:** How does the RL-based pruning policy compare to other pruning baselines in terms of performance?
- **RQ3:** How efficient is the TraceDiag in the production environment, and what is its execution time at each step?

We provide answers to these questions in the following subsections.

5.1 Experiment Setup

We conduct experiments using the `PyTorch` [36] and `Dowhy` [4, 45] frameworks. The experiments are performed on a system running Ubuntu 22.04.1, equipped with an Intel(R) Xeon(R) Gold 6338 CPU consisting of 127 cores with a clock speed of 2.00GHz.

5.1.1 Dataset. For evaluating the performance of TraceDiag, we employ a real dataset obtained from the Microsoft Exchange microservice system, as described in Sec. 3 of this paper. Due to the unavailability of large-scale trace data specifically designed for RCA in the public domain, we rely on this dataset. The dataset encompasses various incident cases, out of which we randomly select 3 cases for training the RL-based pruning policy. The remaining 9 cases are kept separate for the purpose of testing. It is important to emphasize that all procedures involved in data collection strictly adhered to relevant regulations, ensuring the preservation of complete anonymity and desensitization of the datasets. These measures effectively address any potential privacy concerns associated with the utilization of the dataset in this study.

5.1.2 Baselines. We conducted a comparative analysis of the TraceDiag with a broad range of baselines. For the graph pruning phase, exhausted search or grid search is not applicable due to the huge action space. We chose two alternative methods for comparison: random pruning and a heuristic pruning strategy designed by experienced engineers, outlined below:

- **Random:** This method randomly selects pruning actions from the action pool in Table 5 to filter out components that do not meet the requirement. The search continues until the number of nodes in the pruned graph is below a threshold.
- **Heuristic [20]:** The Heuristic method is designed by engineers who have years of experience in the field of RCA. We leverage their expertise to design the heuristic pruning rules based on their domain knowledge.

In terms of the RCA phase, we selected state-of-the-art trace-based and monitoring-based RCA approaches, which leverage correlations and graph theory, as listed below:

- **BackTrace [35]**: This method employs a backward BFS to evaluate abnormality scores. It takes into account two key factors: the strength of path correlation originating from the frontend service and the Pearson correlation coefficient between individual components and the frontend service.
- **GrootRank [49]**: It is a customized algorithm from PageRank [42] to calculate the root cause ranking. It incorporates access distances from the frontend to handle situations where tied results may occur. In this ranking, higher priority is given to root causes with shorter access distances (sum) and vice versa.

5.1.3 Performance Metrics. In order to evaluate the performance of our method, we utilized two metrics to assess the precision of the RCA results: **PR@k**[52] and **RankScore**[35]. **PR@k** measures the number of correct root causes among the top-k predictions and is calculated as follows:

$$\text{PR@k} = \frac{\sum_{i=1}^{k} \text{RC}_i^{\text{pred}} \in \text{RC}^{\text{true}}}{\min\left(\left|\text{RC}^{\text{true}}\right|, k\right)}, \tag{5}$$

where RC^{true} represents the ground-truth list of root causes and RC^{pred} is the set of predicted root causes. Additionally, we compute the average of **PR@k** for $k = 1, 2, \cdots, 5$ as **PR@Avg**. Another metric used to assess the rankings of the RCA is **RankScore**, defined as $= \frac{1}{|\text{RC}^{\text{true}}|} \sum_{v \in \text{RC}^{\text{true}}} (1 - \text{s}(v))$. Here, $\text{s}(v)$ represents $\frac{\text{rank}(v)}{N}$ when $\text{rank}(v) <$ the number of true root causes. Otherwise, $\text{s}(v)$ is assigned as 1. In this context, N denotes the length of the predicted root cause list, and $\text{rank}(v)$ signifies the rank of v. The RankScore metric averages the rank performance for each ground-truth root cause service. Furthermore, to assess the recall quality of the pruned graph, we introduce the **HitRootCause** metric, defined as $= \frac{|RC^{\text{hit}}|}{|RC^{\text{true}}|}$, where $|RC^{\text{hit}}|$ denotes the number of ground-truth root causes appearing in the pruned graph, and $|RC^{\text{true}}|$ represents

Table 6: Comparison of different RCA approaches.

RCA Method	PR@1	PR@3	PR@5	PR@Avg	RankScore
BackTrace	0.222	0.481	0.689	0.484	0.678
GrootRank	0.111	0.259	0.578	0.306	0.492
TraceDiag	**0.667**	**0.852**	**0.956**	**0.834**	**0.818**

the total number of ground-truth root causes. A lower value of HitRootCause indicates that the root cause services are not preserved in the pruned graph, rendering the subsequent RCA infeasible.

5.2 RCA Performance Comparison

Table 6 presents the comprehensive performance evaluation of our proposed TraceDiag approach compared to other baselines using 9 testing cases. All evaluations are performed on the pruned dependency graph obtained from our RL-based approach. Our findings demonstrate the superiority of TraceDiag, which leverages CausalRCA method. It consistently outperforms the other baselines across all performance metrics. Specifically, TraceDiag achieves a significant improvement of at least 72.3% in **PR@Avg** and 20.6% in **RankScore**, indicating its enhanced ability to accurately rank the true root cause higher. A closer examination of the performance reveals that TraceDiag successfully identifies the true root cause as the top-ranked recommendation in 66.7% of cases. Furthermore, its top-5 recommendations encompass 95.6% of the true root causes, which is a remarkable achievement.

These notable improvements highlight the superiority of the CausalRCA approach, showcasing its robustness and accuracy in RCA. In contrast, the correlation-based approach, such as BackTrace, falls short in capturing the complex relationship between service latency and its root cause. Correlation alone is insufficient to explain the latency patterns observed across different services. The presence of confounding factors that contribute to high latency in multiple services can lead to misleading conclusions when relying solely on correlation-based RCA methods. Hence, our findings emphasize the reliability and effectiveness of causal approaches in RCA, aligning with the insights gained from Sec. 3.4.

5.3 Dependency Graph Pruning Comparison

Table 7 provides a comparison of the impact and performance of different graph pruning methods, namely Random, Heuristic, and TraceDiag (RL-based), across 9 testing cases. The RL-based approach demonstrates remarkable convergence efficiency, converging in only 36 episodes despite being trained on only three incidents. On average, TraceDiag retains only 11 components after pruning, which is equivalent to the number of components retained by the Heuristic approach designed by experienced engineers. This represents a significant reduction (98%) compared to the original full dependency graph, which initially consists of over 500 components.

Despite the substantial reduction, the pruned graph retains 92.9% of the root cause services (**HitRootCause**). This indicates that the majority of pruned components are indeed redundant and do not contribute to the RCA process. In contrast, the Heuristic method retains only 83.3% of the root cause services in the final pruned graph. Consequently, performing RCA on the remaining one-sixth of incidents would not yield the root cause, as those components have already been removed from the data.

Table 7: Comparison of different graph pruning approaches.

Method	Node	HitRootCause	PR@5	PR@Avg	RankScore
Random	20	0.500	0.289	0.300	0.355
Heuristic	**11**	0.833	0.756	0.660	0.613
TraceDiag	**11**	**0.929**	**0.956**	**0.834**	**0.818**

The impact of the pruning method is also reflected in the subsequent RCA performance. Our TraceDiag approach achieves a 26.5% improvement in **PR@5**, a 26.3% improvement in **PR@Avg**, and a 33.4% improvement in **RankScore** compared to the Heuristic pruning method. Notably, these improvements surpass those achieved by the Random pruning method in an larger margin. These results demonstrate the superior performance of the RL-based pruning approach in accurately removing redundant components, leading to significant enhancements in the efficiency and effectiveness of the RCA process. Notably, an important advantage of TraceDiag is its ability to generalize well to unseen cases, as it can be trained on only three incidents and still achieve satisfactory performance on the nine testing cases. Moreover, TraceDiag converges rapidly in a small number of trials (episodes), making the pruning process fully automated, efficient, and adaptable to the RCA task.

Finally, we conduct a detailed analysis of the filtering tree learned by our TraceDiag to gain insights into the pruning policy acquired through the RL approach. Fig. 5 illustrates the learned filtering tree, which provides a clear interpretation of the pruning policy. The average number of pruned components across all testing cases for each *FilterOut* action is also show in the figure. The filtering tree consists of a total of 11 pruning actions, with 5 of them being *FilterOut* actions. It is evident that all *FilterOut* actions contribute to the final pruning results by removing redundant components from the dependency graph to varying extents. Furthermore, the pruning actions in the filtering tree encompass all three types of actions specified in the action library (Table 5), indicating their relevance and interpretability in the pruning process. However, we observe that the second *FilterOut* action, which combines the joint filter actions of *RankScore* and *Overhead* derived from TraceArk, removes a significant majority of components on average (81.8%). This finding underscores the pivotal role of anomaly-based pruning, emphasizing the importance of the insights gained in Sec. 3.3.

5.4 Efficiency Evaluation

Furthermore, TraceDiag exhibits high efficiency as an RCA framework. Once the filtering tree is obtained through the RL approach, the execution of the filtering tree on the original trace data has an average runtime of only 1.43 seconds. Subsequently, the subsequent CausalRCA process requires an average of 38.24 seconds, resulting in a total runtime of 39.67 seconds for TraceDiag. This indicates that TraceDiag can achieve excellent RCA performance in large-scale microservice systems with remarkable efficiency. Consequently, it significantly reduces the efforts required by engineers for conducting RCA tasks.

6 DEPLOYMENT & DISCUSSION

We present the production impact of TraceDiag and discuss relevant threats to validity in this section.

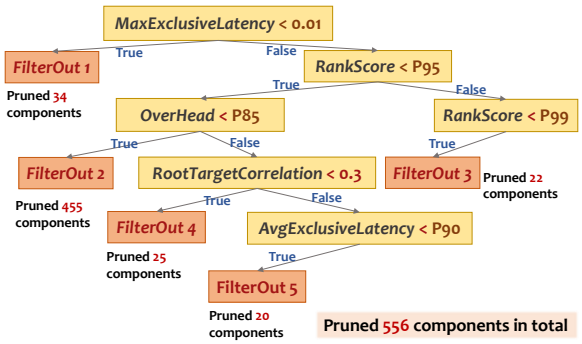


Figure 5: The filtering tree learned by our TraceDiag with averaged pruned components by each FilterOut action.

Table 8: Performance of TraceDiag in real productions.

Framework	PR@Avg	Diagnosis time reduction	
		Easy cases	Hard cases
Heuristic	0.469	50%	96.5%
TraceDiag	0.821		

6.1 Production Impact

The proposed framework, TraceDiag has been integrated as a crucial component within the Microsoft Exchange system to facilitate the localization of root causes in service incidents. Whenever a regression in the end-to-end latency SLA is detected, indicating the occurrence of an incident, TraceDiag is activated and initiates the workflow depicted in Fig. 2. To facilitate the extraction of contrastive patterns, we use one-day data from the incident and utilize the preceding seven days' data as the base window.

Table 8 presents the online performance of TraceDiag in a production environment. Due to confidentiality policies within the company, the specific number of incidents is not disclosed. In comparison to the heuristic RCA approach employed online, the new TraceDiag framework achieves a noteworthy improvement of 75.1% in **PR@Avg**. In incidents where the root cause is easily identifiable, TraceDiag significantly reduces the required RCA time by 50%. For incidents that pose challenges in diagnosing the root cause and necessitate further investigation by engineers, TraceDiag still attains remarkable RCA accuracy while drastically reducing the diagnosis time by over 96.5% compared to manual efforts. These outcomes underscore the exceptional performance of TraceDiag in a production setting, leading to significantly enhanced RCA accuracy and substantial reductions in the time to diagnosis.

6.2 Threats to Validity

Internal threat: The internal threat primarily relates to the accuracy of root cause labels and the implementation of TraceDiag. The root cause labels in our datasets are assigned by experienced domain experts based on incident reports, ensuring their reliability. Moreover, our approach provides the top five candidate root causes, aiding domain experts in decision-making. To mitigate implementation threats, we utilize mature RL frameworks [43], and the code and configurations have been thoroughly reviewed by two authors.
External threat: Our study and experiments are conducted using Microsoft Exchange traces. However, our approach can be easily applied to other systems that have open telemetry traces. While the incidents from the Exchange system may represent a limited range of root causes, considering its extensive size and complexity, we believe our findings are generalizable. In the future, we plan to extend our analyses to encompass a broader range of cloud systems.

7 RELATED WORK

RL for Structure Optimization Previous research has explored the use of RL in discovering and refining neural network structures and various graph scenarios. In the field of neural architecture search (NAS), RL has been employed in AutoML studies [14, 40, 64] to optimize neural network architectures. The RL agent aims to maximize network performance in tasks like image classification or language translation while minimizing computational costs. RL has also gained popularity in graph structure optimization [33, 41]. In social networks [19], RL has been utilized to optimize graph structures for information diffusion [30], community detection [2], and influence maximization [7]. RL agents dynamically add or remove edges between individuals to maximize objectives such as effective information propagation and identification of cohesive communities. In contrast, our TraceDiag framework employs RL to learn a pruning policy for trace data, focusing on service dependency graphs for the purpose of RCA.
Root Cause Analysis with Trace Data Traces are essential in diagnosing system failures in complex cloud systems [18, 22, 27], especially under the large-scale, modern microservice system. Trace-based RCA primarily employs two methods to construct the dependency graph: the topological graph and the causal graph [17]. A topological graph represents the physical or logical service dependency graph and can be either a static graph stored in a Configuration Management Database or a dynamically constructed service call graph [20]. On the other hand, causal graph-based approaches use causal discovery algorithms, such as the PC algorithm [16], to identify relationships among service components. Various inference methods, such as random walk [26] or DFS [8], are then applied to identify root causes. Interested readers may refer to a recent survey [12] for further details. Our approach TraceDiag, which is orthogonal to both topological graph and causal graph-related approaches, focuses on the root cause filtering steps.

8 CONCLUSION

This paper introduces TraceDiag, an end-to-end RCA framework for large-scale microservice systems based on trace data. We design a library of pruning actions derived from an extensive analysis of trace data and the behavior of root cause services, and then leverages RL techniques to autonomously learn an interpretable and adaptive filtering tree that eliminates redundant components from the service dependency graph. This approach significantly enhances RCA efficiency while maintaining RCA performance. The framework further employs a CausalRCA approach, which ensures accurate and robust root cause diagnosis for incidents. Experimental evaluations conducted on real trace datasets validate the efficacy of TraceDiag, highlighting substantial improvements in RCA performance. Notably, TraceDiag has been deployed as a core RCA engine within the Microsoft Exchange microservice system, where it consistently achieves over 75% higher RCA accuracy compared to the legacy RCA framework, and effectively reduces the time required for RCA by up to 96.5%.

REFERENCES

[1] Ahmad Ali, Yanmin Zhu, and Muhammad Zakarya. 2022. Exploiting dynamic spatio-temporal graph convolutional neural networks for citywide traffic flows prediction. *Neural networks* 145 (2022), 233–247.

[2] Mir Mohammad Alipour and Mohsen Abdolhosseinzadeh. 2022. A Multiagent Reinforcement Learning algorithm to solve the Community Detection Problem. *Signal and Data Processing* 19, 1 (2022), 87–100.

[3] Andrea Bernieri, Giovanni Betta, and Consolatina Liguori. 1996. On-line fault detection and diagnosis obtained by implementing neural algorithms on a digital signal processor. *IEEE Transactions on Instrumentation and Measurement* 45, 5 (1996), 894–899.

[4] Patrick Blöbaum, Peter Götz, Kailash Budhathoki, Atalanti A. Mastakouri, and Dominik Janzing. 2022. DoWhy-GCM: An extension of DoWhy for causal inference in graphical causal models. *arXiv preprint arXiv:2206.06821* (2022).

[5] Álvaro Brandón, Marc Solé, Alberto Huélamo, David Solans, María S Pérez, and Victor Muntés-Mulero. 2020. Graph-based root cause analysis for service-oriented and microservice architectures. *Journal of Systems and Software* 159 (2020), 110432.

[6] Kailash Budhathoki, Dominik Janzing, Patrick Bloebaum, and Hoiyi Ng. 2021. Why did the distribution change?. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 1666–1674.

[7] Haipeng Chen, Wei Qiu, Han-Ching Ou, Bo An, and Milind Tambe. 2021. Contingency-aware influence maximization: A reinforcement learning approach. In *Uncertainty in Artificial Intelligence*. PMLR, 1535–1545.

[8] Pengfei Chen, Yong Qi, Pengfei Zheng, and Di Hou. 2014. Causeinfer: Automatic and distributed performance diagnosis with hierarchical causality graph in large distributed systems. In *IEEE INFOCOM 2014-IEEE Conference on Computer Communications*. IEEE, 1887–1895.

[9] Yuhang Chen, Chaoyun Zhang, Minghua Ma, Yudong Liu, Ruomeng Ding, Bowen Li, Shilin He, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. ImDiffusion: Imputed Diffusion Models for Multivariate Time Series Anomaly Detection. *arXiv preprint arXiv:2307.00754* (2023).

[10] Yu Gan, Yanqi Zhang, Kelvin Hu, Dailun Cheng, Yuan He, Meghna Pancholi, and Christina Delimitrou. 2019. Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 19–33.

[11] Clive WJ Granger. 1969. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica: journal of the Econometric Society* (1969), 424–438.

[12] Ruocheng Guo, Lu Cheng, Jundong Li, P Richard Hahn, and Huan Liu. 2020. A survey of learning causality with data: Problems and methods. *ACM Computing Surveys (CSUR)* 53, 4 (2020), 1–37.

[13] Xiaofeng Guo, Xin Peng, Hanzhang Wang, Wanxue Li, Huai Jiang, Dan Ding, Tao Xie, and Liangfei Su. 2020. Graph-based trace analysis for microservice architecture understanding and problem diagnosis. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1387–1397.

[14] Xin He, Kaiyong Zhao, and Xiaowen Chu. 2021. AutoML: A survey of the state-of-the-art. *Knowledge-Based Systems* 212 (2021), 106622.

[15] Lexiang Huang and Timothy Zhu. 2021. tprof: Performance profiling via structural aggregation and automated analysis of distributed systems traces. In *Proceedings of the ACM Symposium on Cloud Computing*. 76–91.

[16] Markus Kalisch and Peter Bühlman. 2007. Estimating high-dimensional directed acyclic graphs with the PC-algorithm. *Journal of Machine Learning Research* 8, 3 (2007).

[17] Mingjie Li, Minghua Ma, Xiaohui Nie, Kanglin Yin, Li Cao, Xidao Wen, Zhiyun Yuan, Duogang Wu, Guoying Li, Wei Liu, et al. 2022. Mining Fluctuation Propagation Graph Among Time Series with Active Learning. In *Database and Expert Systems Applications: 33rd International Conference, DEXA 2022, Vienna, Austria, August 22–24, 2022, Proceedings, Part I*. Springer, 220–233.

[18] Zeyan Li, Junjie Chen, Rui Jiao, Nengwen Zhao, Zhijun Wang, Shuwei Zhang, Yanjun Wu, Long Jiang, Leiqin Yan, Zikai Wang, et al. 2021. Practical root cause localization for microservice systems via trace analysis. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*. IEEE, 1–10.

[19] Zhuoran Li, Xing Wang, Ling Pan, Lin Zhu, Zhendong Wang, Junlan Feng, Chao Deng, and Longbo Huang. 2023. Network Topology Optimization via Deep Reinforcement Learning. *IEEE Transactions on Communications* (2023).

[20] Dewei Liu, Chuan He, Xin Peng, Fan Lin, Chenxi Zhang, Shengfang Gong, Ziang Li, Jiayu Ou, and Zheshun Wu. 2021. Microhecl: High-efficient root cause localization in large-scale microservice systems. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 338–347.

[21] Haifeng Liu, Jinjun Zhang, Huasong Shan, Min Li, Yuan Chen, Xiaofeng He, and Xiaowei Li. 2019. Jcallgraph: tracing microservices in very large scale container cloud platforms. In *Cloud Computing–CLOUD 2019: 12th International Conference, Held as Part of the Services Conference Federation, SCF 2019, San Diego, CA, USA, June 25–30, 2019, Proceedings 12*. Springer, 287–302.

[22] Ping Liu, Haowen Xu, Qianyu Ouyang, Rui Jiao, Zhekang Chen, Shenglin Zhang, Jiahai Yang, Linlin Mo, Jice Zeng, Wenman Xue, et al. 2020. Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 48–58.

[23] Yudong Liu, Xu Zhang, Shilin He, Hongyu Zhang, Liqun Li, Yu Kang, Yong Xu, Minghua Ma, Qingwei Lin, Yingnong Dang, et al. 2022. UniParser: A Unified Log Parser for Heterogeneous Log Data. In *Proceedings of the ACM Web Conference 2022*. 1893–1901.

[24] Chan Li Long, Yash Guleria, and Sameer Alam. 2021. Air passenger forecasting using Neural Granger causal Google trend queries. *Journal of Air Transport Management* 95 (2021), 102083.

[25] Minghua Ma, Yudong Liu, Yuang Tong, Haozhe Li, Pu Zhao, Yong Xu, Hongyu Zhang, Shilin He, Lu Wang, Yingnong Dang, et al. 2022. An empirical investigation of missing data handling in cloud node failure prediction. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1453–1464.

[26] Meng Ma, Jingmin Xu, Yuan Wang, Pengfei Chen, Zonghua Zhang, and Ping Wang. 2020. Automap: Diagnose your microservice-based web applications automatically. In *Proceedings of The Web Conference 2020*. 246–258.

[27] Minghua Ma, Zheng Yin, Shenglin Zhang, Sheng Wang, Christopher Zheng, Xinhao Jiang, Hanwen Hu, Cheng Luo, Yilin Li, Nengjun Qiu, et al. 2020. Diagnosing root causes of intermittent slow queries in cloud databases. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1176–1189.

[28] Minghua Ma, Shenglin Zhang, Dan Pei, Xin Huang, and Hongwei Dai. 2018. Robust and rapid adaption for concept drift in software system anomaly detection. In *2018 IEEE 29th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 13–24.

[29] Elon Mask. [n. d.]. There are 1200 "microservices" server side. ([n. d.]). https://twitter.com/elonmusk/status/1592561366493442050.

[30] Eli Meirom, Haggai Maron, Shie Mannor, and Gal Chechik. 2021. Controlling graph dynamics with reinforcement learning and graph neural networks. In *International Conference on Machine Learning*. PMLR, 7565–7577.

[31] Yuan Meng, Shenglin Zhang, Yongqian Sun, Ruru Zhang, Zhilong Hu, Yiyin Zhang, Chenyang Jia, Zhaogang Wang, and Dan Pei. 2020. Localizing failure root causes in a microservice through causality inference. In *2020 IEEE/ACM 28th International Symposium on Quality of Service (IWQoS)*. IEEE, 1–10.

[32] Abduallah Mohamed, Kun Qian, Mohamed Elhoseiny, and Christian Claudel. 2020. Social-stgcnn: A social spatio-temporal graph convolutional neural network for human trajectory prediction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 14424–14432.

[33] Sai Munikoti, Deepesh Agarwal, Laya Das, Mahantesh Halappanavar, and Balasubramaniam Natarajan. 2022. Challenges and opportunities in deep reinforcement learning with graph neural networks: A comprehensive review of algorithms and applications. *arXiv preprint arXiv:2206.07922* (2022).

[34] Sasho Nedelkoski, Jorge Cardoso, and Odej Kao. 2019. Anomaly detection from system tracing data using multimodal deep learning. In *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 179–186.

[35] Yicheng Pan, Meng Ma, Xinrui Jiang, and Ping Wang. 2021. Faster, deeper, easier: crowdsourcing diagnosis of microservice kernel failure from user space. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 646–657.

[36] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).

[37] Judea Pearl. 1998. Graphical models for probabilistic and causal reasoning. *Quantified representation of uncertainty and imprecision* (1998), 367–389.

[38] Judea Pearl. 2009. *Causality*. Cambridge university press.

[39] Judea Pearl et al. 2000. Models, reasoning and inference. *Cambridge, UK: CambridgeUniversityPress* 19, 2 (2000).

[40] Hieu Pham, Melody Guan, Barret Zoph, Quoc Le, and Jeff Dean. 2018. Efficient neural architecture search via parameters sharing. In *International conference on machine learning*. PMLR, 4095–4104.

[41] Xingyue Pu, Tianyue Cao, Xiaoyun Zhang, Xiaowen Dong, and Siheng Chen. 2021. Learning to learn graph topologies. *Advances in Neural Information Processing Systems* 34 (2021), 4249–4262.

[42] Mark Sanderson. 2010. Christopher D. Manning, Prabhakar Raghavan, Hinrich Schütze, Introduction to Information Retrieval, Cambridge University Press. 2008. ISBN-13 978-0-521-86571-5, xxi+ 482 pages. *Natural Language Engineering* 16, 1 (2010), 100–103.

[43] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).

[44] Lloyd S Shapley et al. 1953. A value for n-person games. (1953).

[45] Amit Sharma and Emre Kiciman. 2020. DoWhy: An End-to-End Library for Causal Inference. *arXiv preprint arXiv:2011.04216* (2020).

[46] Jacopo Soldani and Antonio Brogi. 2022. Anomaly detection and failure root cause analysis in (micro) service-based cloud applications: A survey. *ACM Computing Surveys (CSUR)* 55, 3 (2022), 1–39.

[47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[48] Hanzhang Wang, Phuong Nguyen, Jun Li, Selcuk Kopru, Gene Zhang, Sanjeev Katariya, and Sami Ben-Romdhane. 2019. Grano: Interactive graph-based root cause analysis for cloud-native distributed data platform. *Proceedings of the VLDB Endowment* 12, 12 (2019), 1942–1945.

[49] Hanzhang Wang, Zhengkai Wu, Huai Jiang, Yichao Huang, Jiamu Wang, Selcuk Kopru, and Tao Xie. 2021. Groot: An event-graph-based approach for root cause analysis in industrial settings. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 419–429.

[50] Lu Wang, Chaoyun Zhang, Ruomeng Ding, Yong Xu, Qihang Chen, Wentao Zou, Qingjun Chen, Meng Zhang, Xuedong Gao, Hao Fan, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. Root Cause Analysis for Microservice Systems via Hierarchical Reinforcement Learning from Human Feedback. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA). Association for Computing Machinery, New York, NY, USA, 5116–5125.

[51] Lu Wang, Pu Zhao, Chao Du, Chuan Luo, Mengna Su, Fangkai Yang, Yudong Liu, Qingwei Lin, Min Wang, Yingnong Dang, et al. 2022. NENYA: Cascade Reinforcement Learning for Cost-Aware Failure Mitigation at Microsoft 365. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4032–4040.

[52] Ping Wang, Jingmin Xu, Meng Ma, Weilan Lin, Disheng Pan, Yuan Wang, and Pengfei Chen. 2018. Cloudranger: Root cause identification for cloud native systems. In *2018 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID)*. IEEE, 492–502.

[53] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaojun Chang, and Chengqi Zhang. 2020. Connecting the dots: Multivariate time series forecasting with graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 753–763.

[54] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*. 3634–3640.

[55] Qingyang Yu, Changhua Pei, Bowen Hao, Mingjie Li, Zeyan Li, Shenglin Zhang, Xianglin Lu, Rui Wang, Jiaqi Li, Zhenyu Wu, et al. 2023. CMDiagnostor: An Ambiguity-Aware Root Cause Localization Approach Based on Call Metric Data. In *Proceedings of the ACM Web Conference 2023*. 2937–2947.

[56] Zhengran Zeng, Yuqun Zhang, Yong Xu, Minghua Ma, Bo Qiao, Wentao Zou, Qingjun Chen, Meng Zhang, Xu Zhang, Hongyu Zhang, Xuedong Gao, Hao Fan, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. TraceArk: Towards Actionable Performance Anomaly Alerting for Online Service Systems. In *To appear in Proc. of ICSE*.

[57] Chaoyun Zhang, Marco Fiore, Iain Murray, and Paul Patras. 2021. CloudLSTM: A recurrent neural model for spatiotemporal point-cloud stream forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 10851–10858.

[58] Chaoyun Zhang, Marco Fiore, Cezary Ziemlicki, and Paul Patras. 2020. Microscope: mobile service traffic decomposition for network slicing as a service. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking*. 1–14.

[59] Chaoyun Zhang, Xi Ouyang, and Paul Patras. 2017. ZipNet-GAN: Inferring fine-grained mobile traffic patterns via a generative adversarial neural network. In *Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies*. 363–375.

[60] Chaoyun Zhang, Paul Patras, and Hamed Haddadi. 2019. Deep learning in mobile and wireless networking: A survey. *IEEE Communications surveys & tutorials* 21, 3 (2019), 2224–2287.

[61] Xu Zhang, Chao Du, Yifan Li, Yong Xu, Hongyu Zhang, Si Qin, Ze Li, Qingwei Lin, Yingnong Dang, Andrew Zhou, et al. 2021. Halo: Hierarchy-aware fault localization for cloud systems. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 3948–3958.

[62] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2018. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering* 47, 2 (2018), 243–260.

[63] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. 2019. Latent error prediction and fault localization for microservice applications by learning from system trace logs. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 683–694.

[64] Barret Zoph and Quoc V Le. 2016. Neural architecture search with reinforcement learning. *arXiv preprint arXiv:1611.01578* (2016).

Received 2023-05-18; accepted 2023-07-31